

Software Requirements Specification

C++ Upstox Delivery Trading Bot

Version 1.0 | Prepared on 30 June 2026

This document is a planning and software architecture artifact. It is not financial advice and does not guarantee trading profits. Live trading must be enabled only after dry-run validation, broker compliance checks, and user approval.

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the functional, non-functional, architectural, safety, and compliance requirements for a C++ automated delivery trading bot using the latest Upstox APIs. The system will read company instrument keys from a CSV file, analyze market data using configurable strategies, and automate delivery buy and sell orders after risk validation.

The first implementation shall prioritize dry-run behavior, auditability, safety gates, modular strategy design, and correctness before enabling live order placement.

1.2 Intended Users

- Individual trader or developer operating the bot from an approved Upstox API environment.
- Software developer extending strategy modules in C++.
- System operator reviewing logs, dry-run decisions, and order audit records.

1.3 Scope

The system is limited to Indian equity delivery trading through Upstox. It will not support intraday, margin trading, futures and options, commodities, mutual funds, high-frequency trading, or multi-broker routing in version 1.

1.4 Definitions

Term: Meaning

Delivery: Equity order intended to settle into holdings. Upstox product code D.

Instrument Key: Canonical Upstox identifier such as NSE_EQ|INE002A01018.

LTP: Latest traded price.

OHLCV: Open, high, low, close, volume candle data.

Dry Run: Mode that executes all logic except live order placement.

Strategy: Module that evaluates market data and emits buy or sell signals.

2. Business Objectives

- Automate disciplined delivery buy and sell decisions based on defined strategies.
- Avoid missed profit exits by prioritizing sell orders when target or risk exits trigger.
- Make strategy development modular so RSI divergence, trendlines, breakouts, and future methods can be added cleanly.
- Protect capital through strict risk checks, dry-run defaults, and complete audit logs.

- Stay within Upstox API limits and Indian regulatory constraints.

3. Regulatory And Broker Constraints

The bot shall comply with Upstox and applicable Indian market requirements. As of 30 June 2026, Upstox documentation states separate API limits for order APIs and standard APIs. Order APIs include place, modify, cancel, multi-order, and GTT operations. Regular algos have lower order API throughput than SEBI-registered algos. Upstox also communicates regulatory requirements such as static IP usage for order placement and algo registration thresholds.

- The bot shall run from a configured static IP when live order placement is enabled.
- The bot shall keep live order throughput below the configured Upstox order API limits.
- The bot shall support an optional X-Algo-Name header only when the user has configured an approved algo name.
- The bot shall never bypass OAuth, 2FA, static IP, exchange rules, broker restrictions, or SEBI requirements.
- The bot shall default to dry-run mode and require explicit live-trading confirmation.

4. System Overview

The application shall be a C++20 or C++23 command-line/service application using CMake. It shall be portable across Windows and Linux where practical, with Windows as the first target environment.

4.1 Major Components

- Configuration Loader: loads YAML or JSON configuration.
- CSV Instrument Loader: reads and validates instrument keys and per-stock settings.
- Upstox API Client: handles authenticated REST and WebSocket communication.
- Market Data Service: fetches and streams LTP, OHLCV candles, and volume data.
- Indicator Library: calculates RSI, moving averages, pivots, divergence, trendlines, and support/resistance zones.
- Strategy Engine: runs configured strategy modules and aggregates signals.
- Exit Engine: monitors holdings and triggers target, manual, trailing, stop-loss, or strategy exits.
- Risk Manager: validates every buy and sell decision before an order is queued.
- Order Dispatcher: single controlled component allowed to place, modify, or cancel orders.
- Persistence Store: SQLite database for audit, market snapshots, decisions, and orders.
- Logging System: rotating logs with secret redaction.

5. Operating Modes

Mode: Description

validate: Validates config, CSV, strategy profiles, and risk settings without market calls.

dry-run: Runs market data, strategies, risk checks, and simulated orders without live placement.

paper: Optional future mode that simulates fills and portfolio state.

live: Places real delivery orders after explicit configuration and risk approval.

show-orders: Displays stored orders and decisions from SQLite.

Dry-run shall be the default mode. Live mode shall require mode=live and live_trading_enabled=true in configuration.

6. Input Requirements

6.1 Instrument CSV

The bot shall read target companies from a CSV file. The instrument_key column is mandatory and shall be treated as the canonical identifier. Symbol names are only display metadata.

Minimum CSV columns: instrument_key, symbol, enabled, quantity, max_position_qty, target_profit_pct.

Recommended CSV columns: instrument_key, symbol, exchange, enabled, quantity, max_position_qty, manual_buy_price, manual_target_price, stop_loss_pct, target_profit_pct, trailing_stop_pct, strategy_profile, notes.

6.2 CSV Validation Rules

- instrument_key is mandatory and duplicates shall be rejected.
- enabled must be parseable as true or false.
- quantity and max_position_qty must be positive integers.
- target_profit_pct and stop_loss_pct must be non-negative decimal values when present.
- manual prices must be positive decimal values when present.
- strategy_profile must exist in the configuration when present.
- Invalid rows shall fail startup unless skip_invalid_rows is explicitly enabled.

7. Market Data Requirements

LTP alone is insufficient for RSI divergence and trendline strategies. The system shall support both streaming quotes and historical candle data.

- Fetch latest LTP using Upstox V3 market quote APIs or streaming feeds.
- Fetch OHLCV candles for configurable intervals such as 1 day, 1 hour, and 15 minutes.
- Cache candle data in SQLite to reduce redundant API calls.
- Reject stale, missing, or malformed market data.
- Prefer WebSocket market feeds for live monitoring to reduce REST polling.
- Persist quote/candle snapshots used in every buy or sell decision.

8. Strategy Requirements

8.1 Strategy Framework

The bot shall support modular strategy classes implementing a shared strategy interface. Strategies shall not place orders directly. Strategies shall emit signals that pass through aggregation, risk validation, and order dispatch.

Each signal shall include instrument_key, action, confidence, suggested quantity, suggested entry price, suggested target price, suggested stop loss, reason, strategy name, and timestamp.

8.2 Buy Strategies

- RSI oversold strategy.
- RSI bullish divergence strategy.

- Moving average crossover strategy.
- Trendline support bounce strategy.
- Breakout above resistance strategy.
- Volume breakout confirmation strategy.
- Manual buy price trigger from CSV.

8.3 Sell Strategies And Exit Rules

- Fixed profit target exit, default 10 percent unless overridden.
- Manual target price from CSV or config.
- Strategy-calculated target price.
- Stop loss.
- Trailing stop loss.
- RSI bearish divergence.
- Trendline support break.
- Moving average breakdown.
- Maximum holding period.

8.4 Signal Aggregation

When multiple strategies are enabled, the Strategy Engine shall support any, all, and weighted_score modes. Weighted mode shall compare aggregated confidence against configurable thresholds. Sell signals shall support first_exit_wins so urgent exits are not delayed by unrelated strategies.

9. Exit Priority Requirements

Once a stock is bought, the Exit Engine shall monitor the holding using average buy price, current LTP, manual targets, strategy targets, stop-loss settings, and trailing stop settings.

Exit priority shall be: emergency risk exit, stop loss, manual target, fixed profit target, strategy target, strategy sell signal, trailing stop, maximum holding duration.

If a stock reaches the default 10 percent profit target, the system shall create a high-priority sell decision unless a higher-priority manual or strategy target overrides the target.

10. Risk Management Requirements

- Validate trading mode before order placement.
- Validate stock is enabled and strategy profile is active.
- Validate market session and order window.
- Validate quantity, max position, and daily order count limits.
- Validate max order value and max daily traded value.
- Validate funds before buy orders.
- Validate holdings before sell orders.
- Prevent duplicate open orders for the same instrument and side unless explicitly allowed.
- Provide a kill switch that blocks new live orders immediately.
- Record every risk rejection with a reason code.

11. Order Management Requirements

The bot shall place delivery orders only. Product shall be D, validity shall be DAY, and order type shall default to LIMIT. Market orders shall be disabled by default and available only behind an explicit configuration flag.

Every order shall include instrument_key, transaction type, quantity, price, product, validity, order type, tag, source strategy, and run identifier.

The Order Manager shall track order ID, status, rejection reason, fill information when available, and raw API response metadata. It shall poll or subscribe to order updates until a terminal state or timeout.

12. Multithreading And Concurrency Requirements

The C++ implementation shall use multithreading for responsiveness and computation, but all Upstox API calls shall pass through centralized rate-limited executors. Strategy threads shall never call order APIs directly.

12.1 Required Threads Or Worker Groups

- Market Data Thread: receives WebSocket quote updates and refreshes REST data when needed.
- Strategy Worker Pool: calculates RSI, divergence, trendlines, moving averages, and other indicators.
- Exit Monitor Thread: continuously evaluates holdings for target, stop loss, trailing stop, and strategy exits.
- Order Dispatcher Thread: only component allowed to call order placement, modification, or cancellation APIs.
- Portfolio Sync Thread: refreshes funds, holdings, positions, and order state.
- Persistence Worker: writes logs, decisions, quotes, and order events to SQLite without blocking trading logic.

12.2 Shared State Rules

- Use thread-safe queues for signals, risk events, order requests, and persistence events.
- Use immutable snapshots for market data passed into strategies.
- Use mutexes or read-write locks for shared portfolio state.
- Use atomic flags for shutdown and kill switch state.
- Avoid long-running locks around network calls.

13. API Rate Limit Requirements

The system shall implement token-bucket or leaky-bucket rate limiters for each API category. The configured internal limits shall be lower than broker-published limits to provide safety margin.

API Category: Use

Order API: Place, modify, cancel, multi-order, and GTT operations.

Standard API: Holdings, positions, funds, historical data, and other non-order APIs.

Streaming: Market data and portfolio feeds where available.

Sell orders shall get priority in the order queue, but they shall still respect Upstox limits. The bot shall reserve order capacity for exits so buy orders cannot starve urgent sell orders.

Recommended internal default for regular algo accounts: configure `order_api.safe_requests_per_second` below the published regular algo per-second limit and reserve a majority of that capacity for sell orders.

14. Priority Queue Requirements

The Order Dispatcher shall process order requests by priority and timestamp. Required priority order is: emergency sell or stop loss, target profit sell, manual target sell, strategy sell, buy order, non-urgent modify or cancel.

If multiple sells trigger simultaneously, the dispatcher shall process stop-loss and emergency exits first, then profit exits ordered by urgency and age.

15. Persistence And Audit Requirements

SQLite shall be used for local persistence in version 1. Sensitive tokens shall not be stored in plaintext.

- `bot_runs`: run ID, start time, end time, mode, config hash.
- `instruments`: CSV-loaded instruments and validation status.
- `candles`: OHLCV data used by strategies.
- `quotes`: LTP snapshots.
- `strategy_signals`: raw strategy outputs.
- `decisions`: aggregated buy/sell decisions.
- `risk_events`: approvals and rejections.
- `orders`: order requests, statuses, prices, quantities, and Upstox order IDs.
- `api_events`: redacted request metadata, response codes, retries, and latency.

16. Logging Requirements

- Log startup configuration summary without secrets.
- Log CSV validation summary.
- Log market-data freshness warnings.
- Log every strategy signal and aggregation result.
- Log every risk rejection and approval.
- Log every live and dry-run order decision.
- Redact access tokens, API secrets, and authorization headers.
- Support daily rotation and configurable log levels.

17. Configuration Requirements

Configuration shall be stored in YAML or JSON. Required sections are `app`, `upstox`, `input`, `market_data`, `strategies`, `exit_rules`, `risk`, `rate_limits`, `storage`, and `logging`.

Example configuration keys include `mode`, `live_trading_enabled`, `access_token_env`, `instruments_csv`, `candle_intervals`, `default_target_profit_pct`, `default_stop_loss_pct`, `buy_signal_mode`, `sell_signal_mode`, `min_buy_score`, `max_orders_per_day`, `max_daily_traded_value`, `order_api safe limits`, `sqlite_path`, and `log directory`.

18. Security Requirements

- Access tokens shall be supplied through environment variables, secure prompt, or approved secret storage.
- Tokens and secrets shall never be logged.
- Configuration files should not require plaintext API secrets.
- The bot shall validate that live mode is intentionally enabled.
- The bot shall fail closed when authentication, config, or risk checks fail.

19. Error Handling Requirements

- Retry transient HTTP failures with exponential backoff and jitter.
- Do not retry non-retryable validation, authentication, or rejected-order errors blindly.
- Circuit-break order placement after repeated failures.
- Degrade to no-new-buy mode when market data becomes stale.
- Continue monitoring exits when possible even if a non-critical subsystem fails.
- Persist fatal error context before shutdown.

20. Testing Requirements

- Unit tests for CSV parsing and validation.
- Unit tests for indicator calculations: RSI, moving averages, pivots, divergence, trendlines.
- Unit tests for each strategy module.
- Unit tests for signal aggregation.
- Unit tests for risk rules.
- Unit tests for rate limiter behavior and priority queue ordering.
- Mock API tests for Upstox order payloads and quote parsing.
- Integration tests for dry-run trading loop.
- Manual acceptance test before any live trading.

21. Recommended Technology Stack

Area: Recommendation

Language: C++20 or C++23

Build: CMake

HTTP: libcurl, cpr, or Boost.Beast

WebSocket: Boost.Beast or websocketpp

JSON: nlohmann/json

YAML: yaml-cpp, if YAML config is selected

SQLite: sqlite3

Logging: spdlog

Tests: Catch2 or GoogleTest

22. Proposed Source Layout

app: CLI entrypoint and command routing. core: domain models. infra: API, config, CSV, persistence, logging. indicators: reusable market calculations. strategies: strategy modules. risk: risk manager and kill switch. order: order dispatcher and tracker. concurrency: queues, executors, rate limiters. tests: unit and integration tests.

23. Delivery Phases

- Phase 1: C++ project scaffold, config loader, logging, CSV validation, domain models.
- Phase 2: indicator library and strategy interface with simple strategies.
- Phase 3: signal aggregation, exit engine, and risk manager.
- Phase 4: multithreaded queues, rate limiters, and priority order dispatcher in dry-run mode.
- Phase 5: Upstox market data integration and SQLite audit storage.
- Phase 6: order placement integration behind explicit live-mode gates.
- Phase 7: advanced RSI divergence, trendline detection, backtesting, and performance tuning.

24. Acceptance Criteria

- The bot validates a CSV watchlist and rejects invalid rows with clear errors.
- The bot runs in dry-run mode and records strategy signals, decisions, and simulated orders.
- The bot calculates at least RSI, moving averages, and fixed target exits.
- The bot prioritizes sell orders over buy orders in the dispatcher.
- The bot enforces configured API rate limits in tests.
- The bot refuses live trading unless live mode and explicit confirmation are both configured.
- The bot never logs access tokens or authorization headers.
- The bot has unit tests for strategy, risk, rate limiter, and CSV behavior.

25. Open Decisions

- Choose YAML or JSON for configuration.
- Choose exact first strategy set for version 1.
- Decide whether version 1 runs continuously during market hours or runs scheduled scans.
- Decide whether manual target price always overrides strategy target or only when configured per stock.
- Decide whether market orders are completely forbidden or allowed with an explicit high-risk flag.

26. References

- Upstox Authentication: <https://upstox.com/developer/api-documentation/authentication/>
- Upstox Instruments: <https://upstox.com/developer/api-documentation/instruments/>
- Upstox Market Quote LTP V3: <https://upstox.com/developer/api-documentation/ltp-v3/>
- Upstox Orders: <https://upstox.com/developer/api-documentation/place-order/>
- Upstox Rate Limits: <https://upstox.com/developer/api-documentation/rate-limiting/>
- Upstox WebSocket Docs: <https://upstox.com/developer/api-documentation/websocket/>
- Upstox Regulatory Update: <https://community.upstox.com/t/important-update-regulatory-changes-for-api-and-algo-trading-are-now-live/14874>